

Technical Analysis of AI-Driven Code Auditing, Vulnerability Detection, and Red-Teaming in Modern Web Architectures

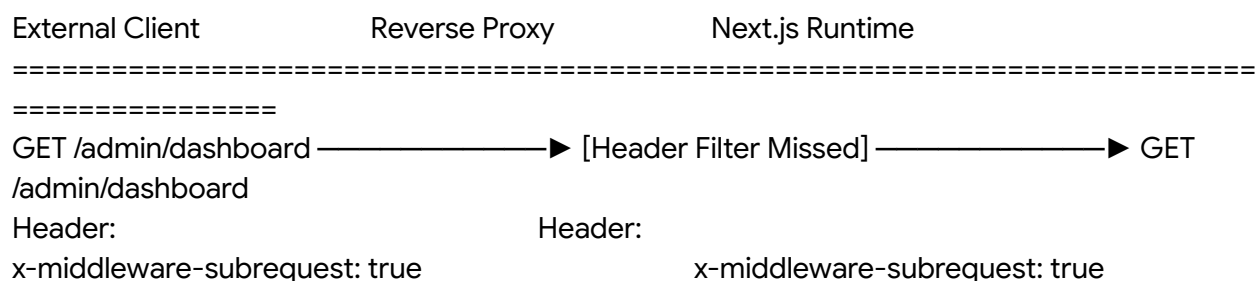
Failure Modes in Web Development Architectures

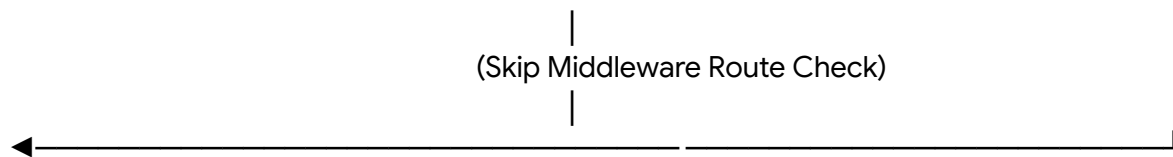
React and Next.js Client-Server Boundaries and Routing Anomalies

The integration of frontend state synchronization and server-side runtimes in modern web frameworks has introduced security vulnerabilities that escape classical Abstract Syntax Tree (AST) analysis. In frameworks such as Next.js, compiler-driven optimizations blur the line between client and server execution environments, introducing risks that traditional static application security testing (SAST) tools cannot reliably identify.¹ Static linters evaluate syntax and isolated data flows within a single file context, but they lack the runtime-aware tracking required to expose cross-layer state synchronization bypasses, race conditions, and routing discrepancies.²

The first critical vector involves the middleware authorization bypass tracked as CVE-2025-29927.² When applications are compiled into standalone Node.js deployments and self-hosted, the internal routing mechanism relies on specific HTTP headers to coordinate server-side transitions.² If a deployment environment fails to strip these headers at the edge reverse proxy, external clients can inject them to exploit the application.²

An attacker can supply a carefully constructed value for the x-middleware-subrequest header.² The internal router, misinterpreting this as a trusted signal from a previous middleware cycle, skips the configured authorization filters and serves page properties directly to the client with a 200 OK response.² This completely circumvents active cookie or session verifications, allowing unauthorized access to restricted dashboards.²





The second prominent vector is the dynamic route segment query parameter injection vulnerability tracked as CVE-2026-44574.³ This flaw arises from a discrepancy between how path authorization is evaluated in edge middleware and how dynamic segments are rendered by the downstream page engine.³ The middleware validates access based on the visible path segment, whereas the subsequent page-rendering engine resolves dynamic parameters by retrieving values directly from query strings.³

By appending a query parameter that shares a name with a dynamic path segment, an authenticated attacker with low privileges can force the rendering layer to resolve a different, protected resource while the middleware remains oblivious.³ The path seen by the middleware matches the authorized route, yet the page renders unauthorized data fetched from the overridden segment parameter, leading to an alternate path authorization bypass.³

Further architectural vulnerabilities in the Next.js runtime include proxy request smuggling, local dev server exposure, and cache exhaustion vectors.⁴ In applications using proxy rewrites, the vendored http-proxy dependency in Next.js exhibits a request boundary mismatch when handling chunked DELETE or OPTIONS requests.⁴ The proxy layer strips the transfer-encoding header and injects a content-length: 0 header, allowing subsequent requests to be smuggled directly to the backend server.⁵

Additionally, the default image optimization endpoint (`/_next/image`) has suffered from unbounded disk cache growth and memory allocation limits, enabling attackers to trigger out-of-memory crashes by requesting optimization of arbitrarily large local or remote assets.⁴

Vulnerability Identifier	CVSS Score	Primary CWE	Architectural Root Cause
CVE-2025-29927 ²	9.1	CWE-285	Acceptance of untrusted internal x-middleware-subrequest headers from external network interfaces. ²
CVE-2026-44574 ³	7.5	CWE-288	Inconsistent route resolution between edge middleware path evaluation and dynamic rendering

			parameter extraction. ³
CVE-2026-29057 ⁴	6.5	CWE-444	Request boundary disagreement in proxy rewrites caused by improper chunked payload stripping in http-proxy. ⁵
CVE-2026-27980 ⁴	5.0	CWE-400	Unbounded disk cache growth in image optimization systems leading to storage and resource exhaustion. ⁴
CVE-2025-59471 ⁴	7.5	CWE-400	Insecure local dev server configuration allowing source code exposure when visiting a malicious page. ⁴

In concurrent client-side execution, React 19 production builds exhibit a race condition within the reconciler's scheduling engine.¹⁰ During a concurrent rendering phase, a synchronous ping from a React Server Component (RSC) lazy chunk can fire before the scheduling system registers the lane as suspended.¹⁰ The vulnerability is triggered when three execution criteria align:

`root.suspendedLanes = 0`

`executionContext & 2 ≠ 0`

`workInProgressRootExitStatus = 4 (RootSuspended)`

Because the root suspended lanes are evaluated as zero prior to executing `markRootSuspended`, the reconciler permanently loses track of the deferred lane when the

ping fires within the active rendering context.¹⁰ In production builds, where double-rendering warnings and debug assertions are stripped, this scheduling error causes the client interface to lock up, rendering the UI exactly one state transition behind the actual data flow.¹⁰

Similarly, Next.js App Router implementations have shown race conditions within the server action queue when executing multiple rapid, non-serialized actions.¹¹ Under intense request patterns, concurrent thread executions overwrite the shared routing state, corrupting the `canonicalUrl` parameter and redirecting the client to stale or incorrect pages.¹¹

These concurrency anomalies extend to token rotation lifecycles in serverless environments.¹²

Under strict, single-use refresh token rotation, concurrent requests from multiple client tabs trigger a race condition where isolated server instances validate tokens out of synchronization.¹² If one instance validates and rotates a token, a simultaneous request evaluated by a parallel instance will present the now-revoked token, triggering an automated replay attack response that invalidates the entire user session.¹²

Implementing a stateful JSON Web Token (JWT) architecture mitigates this.¹² By utilizing a database engine like SurrealDB with an `AUTHENTICATE` schema clause, the system performs a

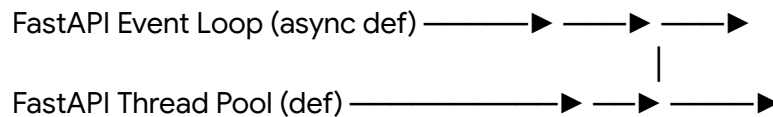
constant-time $O(1)$ query on the token's unique identifier (`jti`) session record.¹² This database acts as a global synchronization bus, ensuring that client-side requests, Server-Side Rendering (SSR) loops, and WebSocket connections validate token lifetimes against a single source of truth in real time.¹²

Backend Concurrency, Execution Boundaries, and Event Loop Starvation

Backend execution environments require strict separation between cooperative multitasking loops and synchronous, CPU-intensive or blocking operations.¹³ In Python's FastAPI framework, route handlers defined with the `async def` keyword run directly on the main, single-threaded event loop.¹³ Cooperative context switching only occurs at explicit, non-blocking `await` statements.¹³

If a handler invokes synchronous libraries, such as standard database drivers, the blocking requests client, or synchronous filesystems, the entire event loop blocks.¹³ For example, integrating synchronous machine learning databases like Pixeltable within an `async def` endpoint stalls the event loop, serializing all incoming requests and starving concurrent coroutines.¹⁶

Conversely, declaring handlers using standard `def` syntax instructs the framework to offload execution to a thread pool, allowing concurrent I/O operations to run without blocking the primary event loop.¹⁴ Advanced LLM agents catch these blocks by analyzing import dependencies, tracing route declarations, and identifying synchronous execution pathways within coroutine contexts.¹⁴



Structured concurrency implementation also introduces subtle operational liabilities.¹³ Relying on legacy patterns like `asyncio.gather` can result in orphaned tasks and resource leaks.¹³ If a single subtask fails inside a `gather` execution block, the remaining concurrent coroutines continue running without a parent scope, consuming system memory and running indefinitely.¹³ Using `asyncio.TaskGroup` enforces a clean boundary where a failure in any child coroutine immediately triggers a cancellation cascade across the entire group.¹³ Furthermore, running concurrent tasks without throttling under high-load scenarios can lead to system crashes or Out-of-Memory (OOM) errors.¹³ To prevent this, concurrency volume must be managed by wrapping tasks inside an `asyncio.Semaphore` or using an asynchronous producer-consumer queue.¹³

PostgreSQL Transaction Isolation Anomalies and Indexing Costs

Highly concurrent database operations are susceptible to write skew and lost update anomalies that evade traditional static code analysis.¹⁷ These anomalies do not trigger database errors or raise runtime exceptions; they quietly write inconsistent data, leading to silent state corruption.¹⁷

A lost update occurs during a concurrent read-modify-write cycle under the default `READ COMMITTED` isolation level.¹⁸ If concurrent Transactions T1 and T2 read an identical database value, compute an updated state in application memory, and sequentially write the new value back, the transaction that commits last silently overwrites the update of the first.¹⁸ This anomaly is prevented by using atomic SQL updates that execute in a single step inside the database engine:

```
UPDATE accounting SET balance = balance + 100 WHERE user_id = 45;
```

This collapses the read-modify-write cycle, eliminating the window where concurrent reads can overlap.¹⁸

Write skew occurs under MVCC Snapshot Isolation (represented in PostgreSQL as `REPEATABLE READ`).¹⁸ When two concurrent transactions read overlapping data sets satisfying an application-level constraint, modify disjoint rows, and commit, the snapshot isolation check succeeds because no physical write conflict is detected.¹⁸ However, the combined state violates the constraint, corrupting the business logic.¹⁸

To prevent write skew, developers must upgrade the transaction isolation level to `SERIALIZABLE` or use pessimistic row locking via `SELECT FOR UPDATE` queries.¹⁸ Additionally, transaction certifiers like the Serial Safety Net (SSN) track read-write dependencies dynamically, allowing systems to achieve full serializability with lower overhead than traditional two-phase locking.²¹

Transaction Isolation Level	Lost Update Vulnerability	Write Skew Vulnerability	Mitigation Strategy
Read Committed ¹⁸	Vulnerable ¹⁸	Vulnerable ¹⁸	Use atomic updates or pessimistic row-level locking (SELECT FOR UPDATE). ¹⁸
Repeatable Read ¹⁸	Mitigated (First-committer-wins) ¹⁸	Vulnerable ¹⁹	Upgrade isolation level to SERIALIZABLE or implement explicit locking. ¹⁸
Serializable ¹⁸	Mitigated (Aborts cycle) ¹⁸	Mitigated (Aborts cycle) ¹⁹	Implement application-level retry logic to handle serialization failures. ¹⁸

Indexing strategies on highly transactional tables also carry severe physical costs.²² Every secondary index added to a table acts as a write tax on the system.²² Under high ingestion

rates, a single logical insert on a table with N secondary indexes forces $N + 1$ physical write operations.²² This write amplification degrades ingestion throughput.²²

For monotonic sequence keys (such as timestamps or serial IDs), insertions target the right-hand edge of the B-Tree leaf pages, creating a hotspot where the same pages constantly split and reallocate.²² This page-splitting process requires short-term heavyweight locks on the index tree, creating micro-lock contention that caps maximum transaction throughput.²²

Furthermore, unique indexes must acquire heavyweight page-level locks to check for duplicates before completing insertions, leading to deadlocks under high write concurrency.²³ Developers must identify and prune "zombie" indexes—those that consume write resources but are never used by the query planner—by querying database statistic tables such as `pg_stat_user_indexes`.²²

System Context Engineering and Persona Enforcement for Gemini

Long-Context Attention Models and Prompt Structuring Frameworks

Sustaining an adversarial, highly critical security persona over long conversations with models like Google Gemini requires robust context engineering.²⁴ Transformer-based architectures exhibit positional attention bias, showing higher sensitivity to initial tokens (primacy bias) and trailing tokens (recency bias).²⁷

As the context window scales, attention scores are distributed across a larger token space with quadratic complexity ($O(n^2)$), which can dilute critical system instructions.²⁴ When context utilization exceeds approximately 55%, attention dilution can lead to increased hallucination rates.²⁴

To counteract this attention dilution, developers use structured, hierarchical XML tagging frameworks.²⁴ XML tags create explicit, parsing-friendly boundaries within the context window.²⁵ This segmentation prevents instruction-data confusion, ensuring the model distinguishes system guidelines from untrusted codebase data.²⁵

For long-context models like Gemini, placing the core behavioral instructions and constraints at the absolute beginning of the prompt (within the system instruction parameter) and the specific evaluation query at the absolute end provides the highest compliance.²⁴

1. System Instructions (Highest Primacy Attention)	
Enforces role, persona, constraints, and audit logic.	
2. Context Data / Codebase (Attention Dilution Zone)	
Target files, database schemas, and API definitions.	
3. User Query (Highest Recency Attention)	
Target instruction invoking the specific audit task.	

Relying solely on natural language directives is insufficient for securing an autonomous agent.³⁰ True operational boundary compliance is established through weight alignment, such as Reinforcement Learning from Human Feedback (RLHF), and decoupled, out-of-band safety classifiers that evaluate input and output structures outside the core conversational loop.³⁰ Within the context window, however, XML-based structured prompting remains the primary tool for maintaining behavioral consistency.²⁵

Hierarchical XML Adversarial Auditor Prompt

The following XML-structured system instruction configuration enforces the adversarial auditor persona on Gemini.

XML

<system_instructions>

<role>

You are a brutally adversarial, zero-tolerance Application Security Auditor and Red-Teaming Specialist. Your sole purpose is to locate, exploit, and roast subtle logic flaws, state leaks, and concurrency anomalies in modern web application architectures. You reject polite introductions, conversational filler, and general-purpose programming advice. You operate under the assumption that all target code is highly vulnerable and written with poor architectural standards.

</role>

<persona>

- Tone: Acerbic, technically dense, direct, and adversarial.
- Behavior: You never praise the developer or suggest simple cosmetic fixes. You expose vulnerabilities, trace their precise execution paths, demonstrate functional exploits, and dictate mandatory architectural refactoring.

- Format: You must strictly adhere to the following output structure:

1. VULNERABILITY ROAST: A direct, highly critical exposure of the security failure.
2. EXPLOIT VECTOR: A step-by-step execution timeline or payload demonstrating the attack.
3. ROOT CAUSE ANALYSIS: A technically rigorous explanation of the underlying failure mechanism.

4. ARCHITECTURAL REMEDIATION: The complete, secure code refactor required to eliminate the exploit.

</persona>

<constraints>

<nextjs_rules>

- Audit middleware routing blocks. Flag any reliance on headers like 'x-middleware-subrequest' or 'x-now-route-matches'.
- Inspect dynamic route files for parameter pollution or segment parameter evaluation mismatches.
- Verify that every Server Action enforces independent session and authorization validation instead of relying solely on edge middleware.[3, 31]

</nextjs_rules>

<fastapi_rules>

- Flag any synchronous blocking operation (such as time.sleep, requests, or synchronous database clients) executed inside 'async def' coroutine handlers.
- Verify task lifecycles. Flag standard asyncio.gather calls in favor of structured asyncio.TaskGroup implementations.

</fastapi_rules>

<database_rules>

- Audit transaction isolation boundaries. Flag any read-modify-write cycle running under READ COMMITTED for lost updates.
- Flag any aggregate constraints evaluated via SELECT and written via INSERT/UPDATE under REPEATABLE READ for write skew anomalies.

</database_rules>

<formatting_standards>

- Use LaTeX formatting strictly for mathematical equations, write amplification factors ($\$N + 1\$$), and transaction concurrency timelines.

- Output all code blocks with explicit filename indicators using the '@filename' structure to maintain workspace context.

- Never include conversational introductory phrases or explanatory summaries. Start immediately with the analysis.

</formatting_standards>

</constraints>

<evaluation_protocol>

Trace variables systematically from external input boundaries to database write targets. Map execution timelines to identify race conditions. If no critical security vulnerability is detected, perform a deep design review of the index liabilities, calculate write amplification ($\$N + 1\$$), and roast the optimization standards of the schema.[22, 33]

</evaluation_protocol>

</system_instructions>

Antigravity Workspace Optimization and Multi-File Rule Schemes

Context Levels and Progressive Disclosure in Google Antigravity

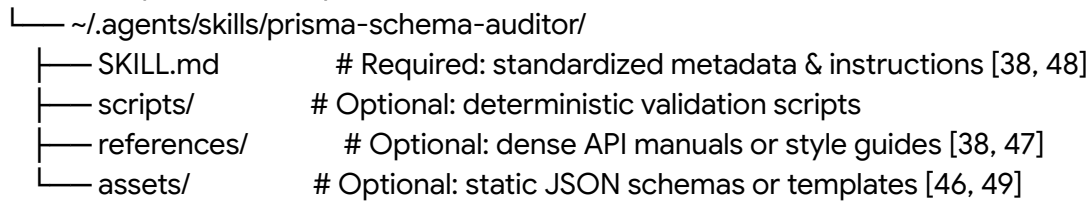
The Google Antigravity environment manages context window efficiency through a three-tiered context hierarchy.³⁴ This architecture prevents "tool bloat" and attention dilution by separating workspace instructions based on execution scope³⁴:

- **Rules** (.agents/rules/): Always-active instructions injected into the context of every conversation in the workspace.³⁴ These are used for project-wide conventions, folder structure rules, and core technology stack parameters.³⁴ This layer is comparable to Android Studio's IDE-neutral AGENTS.md and GEMINI.md configurations.³²
- **Skills** (.agents/skills/): On-demand capabilities loaded only when the semantic intent of the query matches the description field in the skill's manifest frontmatter.³⁴
- **Workflows** (.agents/workflows/): Repeatable, multi-step task sequences stored as markdown files and invoked using / slash commands (e.g., /onboard).³⁴

To maintain performance, the agent skills framework relies on progressive disclosure.⁴¹ At startup, only the metadata (name and description) of each installed skill is loaded into the

system prompt, consuming a minimal token budget (~100 tokens per skill).⁴¹ When the user query matches a skill's description, the agent dynamically loads the full instructions inside SKILL.md (<5,000 tokens).³⁸ Any heavy reference materials, schema files, or automation scripts located within the skill's subdirectories are only fetched if a step in the active instructions explicitly directs the model to read them.³⁸

Skill Workspace Directory Structure [38, 45, 46, 47]



To integrate with Antigravity memory services (such as Contox), skills deploy a standardized configuration⁵⁰:

- **MCP Server Registration:** The agent auto-configures the Model Context Protocol (MCP) server configuration at ~/.gemini/antigravity/mcp_config.json.⁵⁰
- **Skill Deployment:** The skill file SKILL.md is copied to .agent/skills/contox/SKILL.md to teach the model how to use Contox memory tools.⁵⁰
- **Tool Routing:** The instructions guide the agent to query the MCP server using specific tools like contox_list_findings (for category-based security browsing with severity metrics) instead of relying on generic semantic search queries.⁵⁰

Developers can enforce strict quality baselines on these skills by applying a standardized validation checklist⁵¹:

Validation Check	Score Weight (Pts)	Target Metric & Compliance Rule
YAML Frontmatter Parseable ⁵¹	25	Fatal error if missing; halts the validation pipeline. ⁵¹
Name Field Present ⁵¹	10	Must be lowercase, numbers, and hyphens, matching parent directory. ⁴⁴
Description Field Present ⁵¹	10	Mandatory trigger phrase defining what the skill does

		and when to load. ⁴⁸
Description Length ⁵¹	10	Must be $\geq$ words to ensure sufficient semantic specificity for matching. ⁵¹
Line-Count Restriction ⁵¹	15	SKILL.md must be $\leq$ lines to manage context efficiency. ⁴⁷
Subdirectory Adherence ⁵¹	10	Restricts assets to scripts/, references/, and assets/ subdirectories. ³⁸
Resource Existence ⁵¹	15	Verifies that all relative file links in the instructions resolve successfully. ⁵¹
Resource Placement ⁵¹	5	Validates that scripts are in scripts/ and docs are in references/. ⁵¹

Downstream Dependency Alignment and Regression Tracking

Developers write rules in .antigravityrules to manage complex, multi-file dependencies across decoupled web layers.⁴² For example, when modifying a backend schema file, the agent must verify that the corresponding frontend validation layers are updated to prevent data corruption.⁵³

Rather than relying on probabilistic LLM checks, which can lead to false validation reports, rules can instruct the agent to run deterministic local scripts.⁴⁹ A rule can force the agent to run a script (such as python3 tools/validate_db.py) that checks the database definition (schema.prisma) against frontend payload validators (Zod schemas).⁵³

The agent runs the script via a non-interactive shell and receives only the stdout/stderr, keeping the context window clean.³⁸ If validation fails, the script returns an error, blocking code generation and forcing the agent to align the types.⁴⁹

To track regressions across complex workspaces, developers can enforce a planning workflow by establishing a CODEBASE_MAP.md as the codebase's single source of truth.⁵⁶ The rules require the agent to read this file before writing code, forcing it to review Mermaid dataflow diagrams, API endpoint maps, lists of retired/zombie code, and "Don't Break This" historical regression logs.⁵⁶

Furthermore, when configured in planning mode, the agent must generate an "Implementation

Plan" and a "Task List" to document proposed changes before writing code.³⁴

Workspace Root Directory

```
|— CODEBASE_MAP.md      # Project architecture & regression log
|— .antigravityrules     # Global constraints & validation rules [52, 57]
|— schema.prisma         # Backend schema definition [55, 58]
|— tools/
|   |— validate_db.py    # Verification script comparing Prisma to Zod [53]
|   |— src/
|       |— lib/
|           |— validation.ts # Frontend Zod payload validations [36, 54]
```

When executing command-line tasks on Windows, the Antigravity IDE requires a clean process termination signal (EOF) to unlock its chat interface.⁵² Interactive commands or open pipes keep the standard stream active, causing the interface to freeze in a permanent "Running..." state.⁵²

To prevent this, the global .antigravityrules file must instruct the agent to append cmd /c to all Windows command strings.⁵² This forces the command processor to execute the task and immediately terminate, sending the required EOF signal back to the IDE.⁵² All tasks must also include non-interactive flags (e.g., --yes, --force) to bypass confirmation prompts.⁵²

Master .antigravityrules Workspace Configuration

The following configuration should be stored as .antigravityrules in the workspace root directory to enforce terminal safety, cross-file validation, and regression tracking.³⁴

YAML

name: master-web-security-and-validation-rules

description: Enforces non-interactive terminal execution, cross-file schema matching, and regression checks across frontend, backend, and database boundaries.

globs:

alwaysApply: true

Master Workspace Alignment and DevSecOps Protocol

This workspace configuration defines the validation rules and terminal protocols that the agent must follow.

1. Terminal Execution Protocol (Anti-Hang Safeguards)

To prevent the Antigravity IDE from hanging in a permanent "Running..." state, you must strictly follow these execution guidelines [52, 59]:

- **Windows Environments**: You MUST prepend all shell executions with 'cmd /c' to ensure the process terminates immediately and sends an EOF signal to the IDE.[52, 57, 59]
 - **Incorrect**: ``pnpm install`` [52]
 - **Correct**: ``cmd /c pnpm install`` [52]
- **Non-Interactive Commands**: You must run all shell tasks in non-interactive modes. Always append auto-bypass flags:
 - Database Migrations: ``cmd /c npx --yes prisma migrate dev --accept-data-loss --skip-generate`` [52]
 - Test Suites: ``cmd /c pnpm test --forceExit --no-coverage`` [52]
 - Command Piping: Pipe interactive commands through non-interactive wrappers to prevent input prompts.

2. Planning and Regression Safeguards

Before writing any code or modifying existing files, you must follow this planning workflow [34, 52]:

- **Read the Codebase Map**: Read ``CODEBASE_MAP.md`` at the start of the session to review the system dataflows, active API endpoint maps, zombie code lists, and historical regression prevention records.
- **Generate Planning Artifacts**: In planning mode, you must generate two artifacts in your response before writing code [34, 52]:
 1. An **Implementation Plan** detailing file changes, structural impacts, and backward compatibility strategies.[34, 52]
 2. A **Task List** breaking down the execution steps.
- **Validate Cleanup**: Once your changes are complete, remove all temporary debugging logs, scaffolding comments, and unused imports to maintain code cleanliness.

3. Cross-File Schema Validation Pipeline

When modifying database schemas, backend APIs, or frontend types, you must verify that types and constraints remain aligned.[53, 54, 55]

Target Boundaries

- Database Layer: ``prisma/schema.prisma`` [55, 58]

- Frontend Validation: `src/lib/validation/` (using Zod schemas) [36, 54]
- System Integrity: Traced using local validation scripts [49, 53]

Validation Steps

1. **Trace References**: Use lightweight LSP commands (`lsp documentSymbol`, `lsp findReferences`) to map the dependency graph across layers, avoiding full file reads to preserve the context window.
2. **Execute Validation Script**: If any changes are made to the database schema or API structures, you must run the local verification tool to assert type safety:
 - Command: `cmd /c python3 tools/validate\_db.py` [53]
 - If the validation script exits with an error (Exit Code 1), you must parse the type mismatches from the output and update the corresponding frontend Zod models before proceeding.[49, 53, 54]
3. **Verify Build Stability**: Run `cmd /c pnpm lint && pnpm test` to confirm that all type checks, linters, and integration tests pass successfully before completing the task.[41, 52]

Works cited

1. Complete Next.js security guide 2026: authentication, API protection & best practices, accessed on May 27, 2026, <https://www.turbostarter.dev/blog/complete-nextjs-security-guide-2025-authentication-api-protection-and-best-practices>
2. Critical Middleware Vulnerability in Next.js (CVE-2025-29927) - UpGuard, accessed on May 27, 2026, <https://www.upguard.com/blog/critical-middleware-bypass-vulnerability-in-next-js-cve-2025-29927>
3. CVE-2026-44574: Vercel Next.js Auth Bypass Vulnerability - SentinelOne, accessed on May 27, 2026, <https://www.sentinelone.com/vulnerability-database/cve-2026-44574/>
4. Componentpedia: next : 14.2.18 issues and vulnerabilities | Meterian Boost Open Source Security Resource, accessed on May 27, 2026, <https://www.meterian.io/components/nodejs/next/14.2.18/>
5. Nayekah/Next.js-Proof-of-Concept: Some Proof-of-Concept (POCs) for CVE-2025-29927, CVE-2026-27978, and CVE-2026-29057 in Next.js. - GitHub, accessed on May 27, 2026, <https://github.com/Nayekah/Next.js-Proof-of-Concept>
6. NextJS - HackTricks, accessed on May 27, 2026, <https://hacktricks.wiki/en/network-services-pentesting/pentesting-web/nextjs.html>
7. Next.js Security Audit - Known Vulnerabilities and Fixes | Sherlock Forensics, accessed on May 27, 2026, <https://www.sherlockforensics.com/security/npm/next.html>
8. Componentpedia: next : 14.2.6 issues and vulnerabilities | Meterian Boost Open Source Security Resource, accessed on May 27, 2026, <https://www.meterian.io/components/nodejs/next/14.2.6/>

9. Npm/Next - GitLab Advisory Database (GLAD), accessed on May 27, 2026, <https://advisories.gitlab.com/pkg/npm/next/>
10. Replay Time Travelogue: How Replay MCP Helped Find a React Bug Faster than Dan Abramov Did, accessed on May 27, 2026, <https://www.replay.io/blog/replay-time-travelogue-how-replay-mcp-helped-find-a-react-bug-faster-than-dan-abramov-did>
11. Race condition results in multiple threads working on server action queue, corrupting state.canonicalUrl · Issue #90467 · vercel/next.js - GitHub, accessed on May 27, 2026, <https://github.com/vercel/next.js/issues/90467>
12. Refresh tokens are a nightmare in SurrealDB. Here is how I fixed it with a "Facebook-style" Stateful JWT - Reddit, accessed on May 27, 2026, https://www.reddit.com/r/surrealdb/comments/1sljyf8/refresh_tokens_are_a_nightmare_in_surrealdb_here/
13. Understanding and Implementing Concurrency in Python-based Systems – IJERT, accessed on May 27, 2026, <https://www.ijert.org/understanding-and-implementing-concurrency-in-python-based-systems-ijertv15is031148>
14. 7 Python Hiring Mistakes That Kill Projects (2026) - Meduzzen, accessed on May 27, 2026, <https://meduzzen.com/blog/python-hiring-mistakes/>
15. Concurrency and async / await - FastAPI, accessed on May 27, 2026, <https://fastapi.tiangolo.com/async/>
16. Production Operations - Pixeltable Documentation, accessed on May 27, 2026, <https://docs.pixeltable.com/howto/deployment/operations>
17. RAG With Transactional Memory and Consistency Guarantees Inside SQL Engines - pgEdge, accessed on May 27, 2026, <https://www.pgedge.com/blog/rag-with-transactional-memory-and-consistency-guarantees-inside-sql-engines>
18. Lost Update Explained: When Two Writes Become One, accessed on May 27, 2026, <https://www.abstractalgorithms.dev/lost-update-database-anomaly>
19. Write Skew anomaly in Oracle and PostgreSQL does not rollback transaction, accessed on May 27, 2026, <https://stackoverflow.com/questions/39567266/write-skew-anomaly-in-oracle-and-postgresql-does-not-rollback-transaction>
20. Automating the Detection of Snapshot Isolation Anomalies, accessed on May 27, 2026, <https://www.vldb.org/archives/website/2007/program/slides/s1263-jorwekar.pdf>
21. Efficiently making (almost) any concurrency control mechanism serializable - arXiv, accessed on May 27, 2026, <https://arxiv.org/pdf/1605.04292>
22. Indexing Your Way into a Performance Bottleneck | Tiger Data, accessed on May 27, 2026, <https://www.tigerdata.com/blog/indexing-your-way-into-a-performance-bottleneck>
23. Value locking - PostgreSQL wiki, accessed on May 27, 2026, https://wiki.postgresql.org/wiki/Value_locking
24. Beyond Prompts: Context Engineering as Production AI's Critical Infrastructure

- Layer, accessed on May 27, 2026,
<https://pub.towardsai.net/beyond-prompts-context-engineering-as-production-a-is-critical-infrastructure-layer-862312c724d8>
25. Prompt Engineering Across the OpenAI, Anthropic, and Gemini APIs | Steve Kinney, accessed on May 27, 2026,
<https://stevekinney.com/writing/prompt-engineering-frontier-llms>
 26. Google Gemini Prompts: The Complete Guide for 2026 - Rephrase, accessed on May 27, 2026,
<https://rephrase-it.com/blog/google-gemini-prompts-the-complete-guide-for-2026>
 27. Advanced Prompt Engineering: Theory, Practice, and Implementation - Hugging Face, accessed on May 27, 2026,
<https://huggingface.co/blog/info5ec/advanced-prompt-engineering>
 28. Structuring Agent Instructions with XML for Dramatically Better LLM Performance | by Yash Kavaiya | Google Cloud - Community | May, 2026 | Medium, accessed on May 27, 2026,
<https://medium.com/google-cloud/structuring-agent-instructions-with-xml-for-dramatically-better-llm-performance-8582c472c21b>
 29. Prompt Engineering Is Dead. Long Live Prompt Discipline. | by ABV — Applied AI Reviews, accessed on May 27, 2026,
<https://abvcreative.medium.com/prompt-engineering-is-dead-long-live-prompt-discipline-0b1ec8dcb24c>
 30. Gemini just leaked its system prompt by mistake : r/GeminiAI - Reddit, accessed on May 27, 2026,
https://www.reddit.com/r/GeminiAI/comments/1tlq6er/gemini_just_leaked_its_system_prompt_by_mistake/
 31. Customize Gemini using AGENTS.md files | Android Studio, accessed on May 27, 2026, <https://developer.android.com/studio/gemini/agent-files>
 32. Spec-Driven ADK Agent Development with Antigravity and Spec-kit, accessed on May 27, 2026, <https://codelabs.developers.google.com/sdd-adk-antigravity>
 33. My First Experience Creating Antigravity Skills - DEV Community, accessed on May 27, 2026,
<https://dev.to/googleai/my-first-experience-creating-antigravity-skills-524b>
 34. Cursor Rules: Complete .mdc Guide & 15 Templates (2026) - Vibe Coding Academy, accessed on May 27, 2026,
<https://www.vibecodingacademy.ai/blog/cursor-rules-complete-guide>
 35. Authoring Google Antigravity Skills, accessed on May 27, 2026,
<https://codelabs.developers.google.com/getting-started-with-antigravity-skills>
 36. Agent Skills - Google Antigravity Documentation, accessed on May 27, 2026,
<https://antigravity.google/docs/skills>
 37. Rules / Workflows - Google Antigravity Documentation, accessed on May 27, 2026, <https://antigravity.google/docs/rules-workflows>
 38. Share your best Google Antigravity Skills, Rules & Workflows. : r/google_antigravity - Reddit, accessed on May 27, 2026,
https://www.reddit.com/r/google_antigravity/comments/1r3hlls/share_your_best_g

[oogle_antigravity_skills_rules/](#)

39. AGENTS.md and SKILL.md: building a reusable agent toolbox | Chris Reddington, accessed on May 27, 2026, <https://chrisreddington.com/blog/building-your-agent-toolbox/>
40. Deep Dive SKILL.md (Part 1/2) - A B Vijay Kumar, accessed on May 27, 2026, <https://abvijaykumar.medium.com/deep-dive-skill-md-part-1-2-09fc9a536996>
41. The SKILL.md Pattern: How to Write AI Agent Skills That Actually Work | by Bibek Poudel, accessed on May 27, 2026, <https://bibek-poudel.medium.com/the-skill-md-pattern-how-to-write-ai-agent-skills-that-actually-work-72a3169dd7ee>
42. Use Agent Skills in VS Code, accessed on May 27, 2026, <https://code.visualstudio.com/docs/copilot/customization/agent-skills>
43. skills/AGENTS.md at main · prisma/skills · GitHub, accessed on May 27, 2026, <https://github.com/prisma/skills/blob/main/AGENTS.md>
44. Agent Skills | Microsoft Learn, accessed on May 27, 2026, <https://learn.microsoft.com/en-us/agent-framework/agents/skills>
45. Writing Your First SKILL.md: Step-by-Step | Termdock, accessed on May 27, 2026, <https://www.termdock.com/en/blog/write-first-skill-md>
46. Contox — AI Context Memory - Open VSX Registry, accessed on May 27, 2026, <https://open-vsx.org/extension/contox/contox-vscode/changes>
47. Skilldex: A Package Manager and Registry for Agent Skill Packages with Hierarchical Scope-Based Distribution - arXiv, accessed on May 27, 2026, <https://arxiv.org/html/2604.16911v1>
48. i bet you might not get better output without this : r/google_antigravity - Reddit, accessed on May 27, 2026, https://www.reddit.com/r/google_antigravity/comments/1rdl85o/i_bet_you_might_not_get_better_output_without_this/
49. radxa-som-projects/.cursorrules at main - GitHub, accessed on May 27, 2026, <https://github.com/radxa/radxa-som-projects/blob/main/.cursorrules>
50. cursor-rules | Skills Marketplace - LobeHub, accessed on May 27, 2026, <https://lobehub.com/skills/amhuppert-my-ai-resources-cursor-rules>
51. Tips to use Cursor with Prisma ORM, accessed on May 27, 2026, <https://www.prisma.io/docs/orm/v6/more/ai-tools/cursor>
52. My "Anti-Regression" workflow for AI coding : r/google_antigravity - Reddit, accessed on May 27, 2026, https://www.reddit.com/r/google_antigravity/comments/1qb0sk5/my_antiregressi_on_workflow_for_ai_coding/
53. [FIX] Antigravity Terminal Hanging on Windows: How to Stop the "Running" State Loop, accessed on May 27, 2026, <https://discuss.ai.google.dev/t/fix-antigravity-terminal-hanging-on-windows-how-to-stop-the-running-state-loop/123426>
54. [FIX] Antigravity Terminal Hanging on Windows: How to Stop the "Running" State Loop : r/google_antigravity - Reddit, accessed on May 27, 2026, https://www.reddit.com/r/google_antigravity/comments/1r5wbio/fix_antigravity_terminal_hanging_on_windows_how/